

OBJECT ORIENTED PROGRAMMING



Class

- Class is way to bind the data and its associated functions together.
- It allows the data to be hidden if necessary from external use.
- A class can be thought of as a template from which objects are made.
- Class is an user defined data type.
- It has got its own functions or method.
- Class is an entity that has got collection of objects.
 - *e.g a class of Student, Employee, Customer, Course, payment, campus, book etc.*



Object

- An object is an instance of class or a type variable of class.
- Examples for an object are student no, student name etc..
- Once an instance of class is created, it has an identity (address) and a state (value that it holds).
- Although an object's state may change throughout a program, its identity never does.



Encapsulation

- ▶ Wrapping up or binding of data and methods into a single entity is called as Encapsulation.
- ▶ Programmer can hide or encapsulate, some fields or methods of a class, while exposing others.
- ▶ Since the fields or data of a class are typically encapsulated within a class, encapsulation is sometimes referred to as Data Hiding.
- ▶ This makes it easier to update the code for a method within a class because you only need to change the classes that need upgrading.
- ▶ Data is not accessible to the outside world and only those functions which are wrapped inside the class can access it.



Polymorphism

- Polymorphism means “many forms” in Greek.
- It allows the same program instruction to mean different things in different contexts.
- A single function name can be used to handle different number and different types of arguments.
- For example, operator + can be used to generate the sum, for two numbers. But if they are 2 strings, operator + will produce a third string by concatenation.



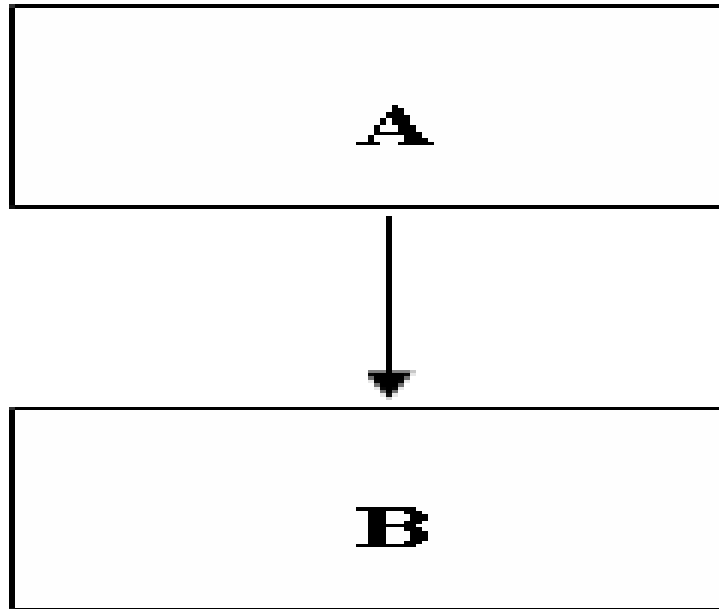
Inheritance

- ▶ Inheritance is a property by which it can derive new class from an existing class.
- ▶ Advantages of Inheritance are
 - ▶ Reusability
 - ▶ Saves time
 - ▶ Saves effort of programmer
- ▶ Different types of Inheritance
 1. Single level Inheritance
 2. Multi level Inheritance
 3. Multiple Inheritance
 4. Hybrid Inheritance
 5. Hierarchical Inheritance



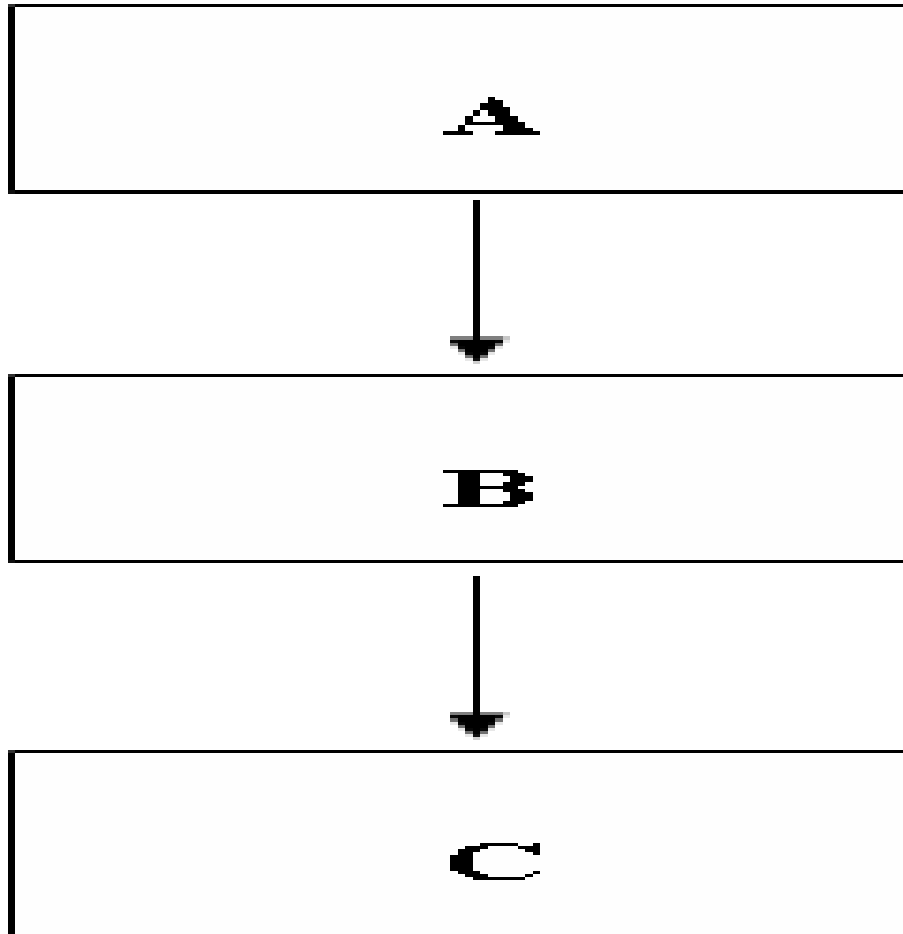
Inheritance (conti..)

1. Single Level Inheritance



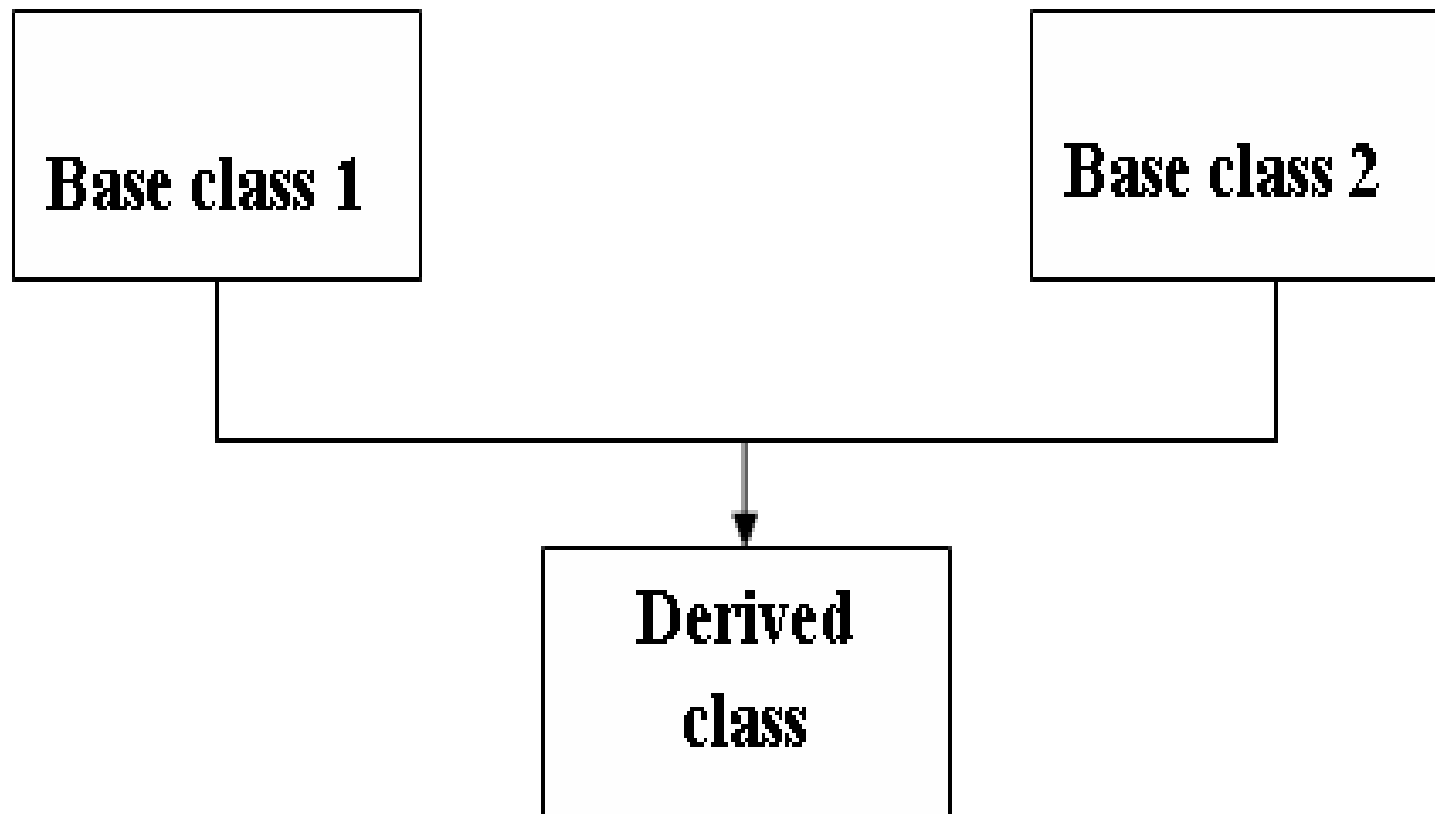
Inheritance (conti..)

2. Multi Level Inheritance



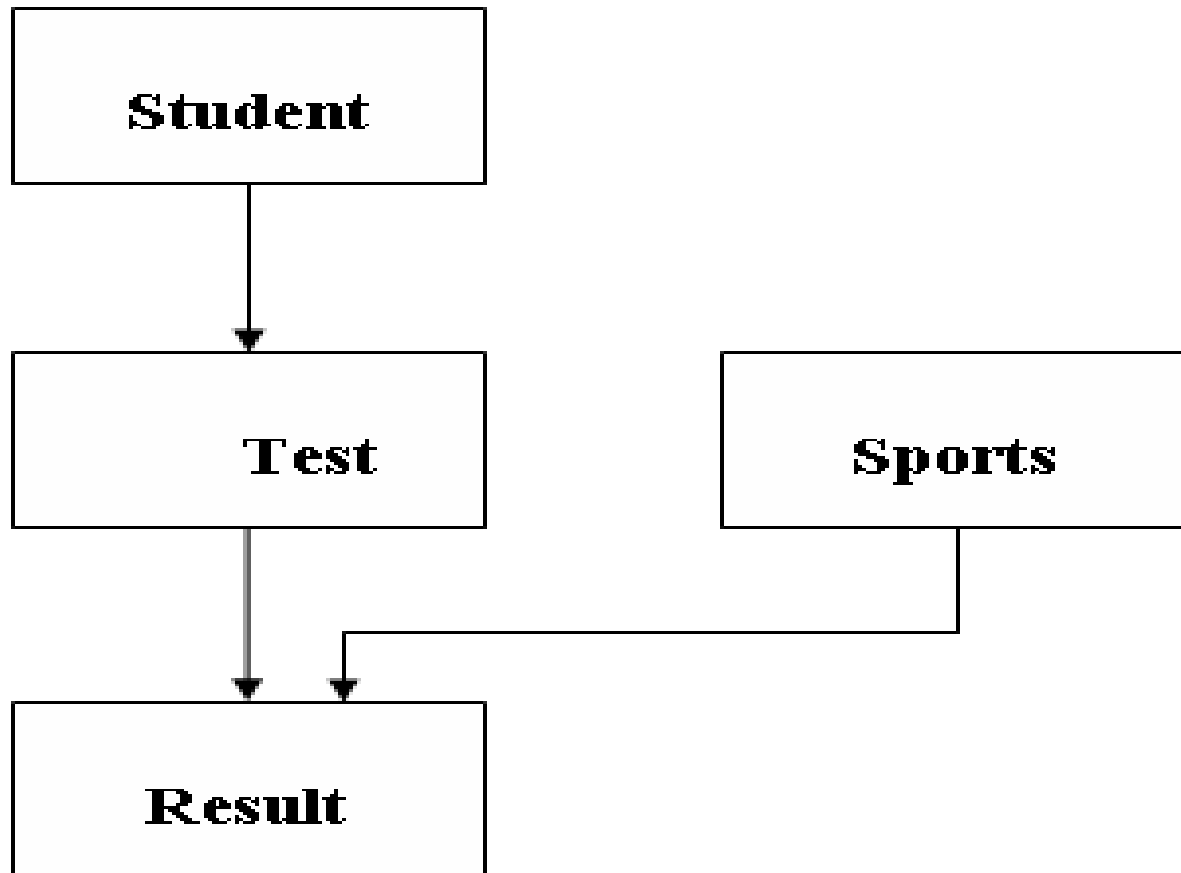
Inheritance (conti..)

3. Multiple Inheritance



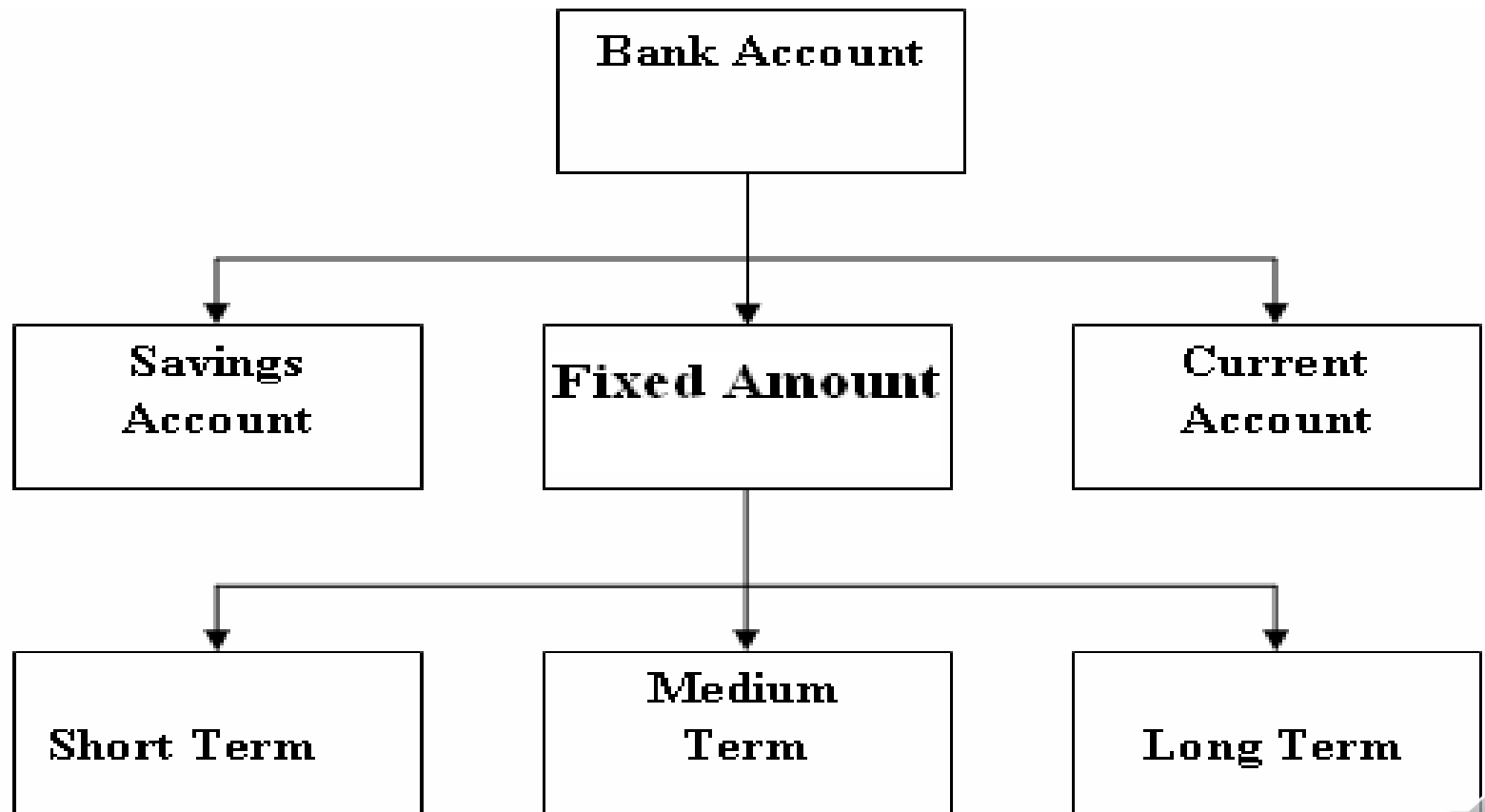
Inheritance (conti..)

4. Hybrid Inheritance



Inheritance (conti..)

5. Hierarchical Inheritance



Relationship between classes (contd.)

- Aggregation

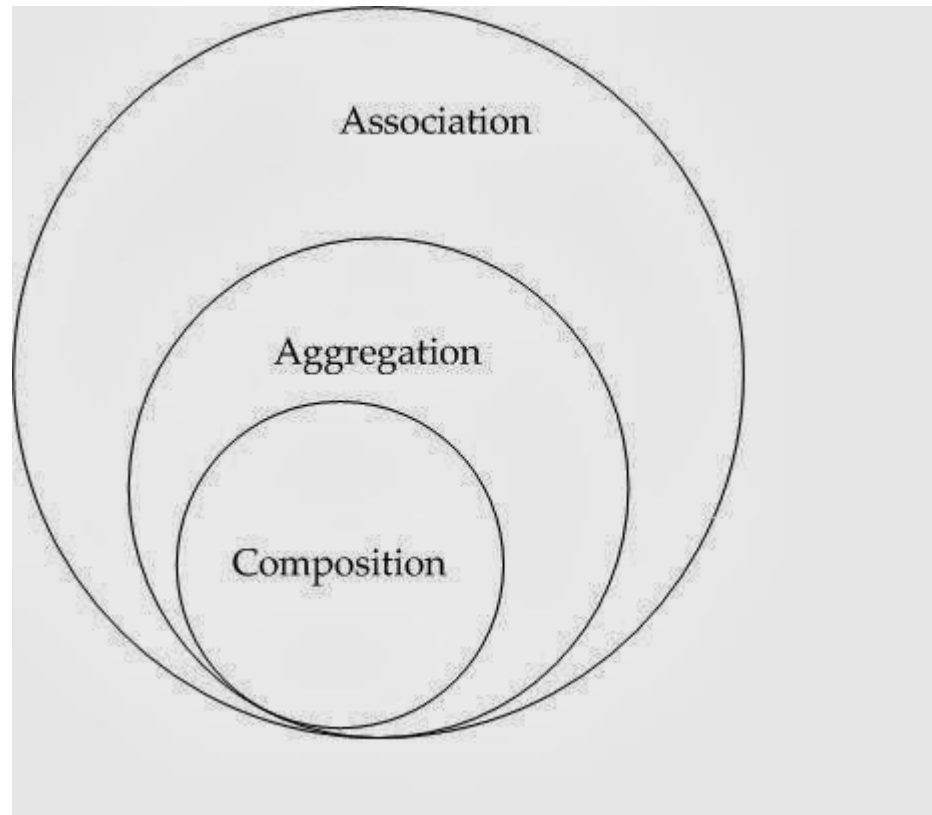
- If a part class can exist, without Whole class, then we will call that Association as Aggregation.
- e.g a car can have passengers who can come and go, so passengers can exist even if the car gets destroyed.

- Composition

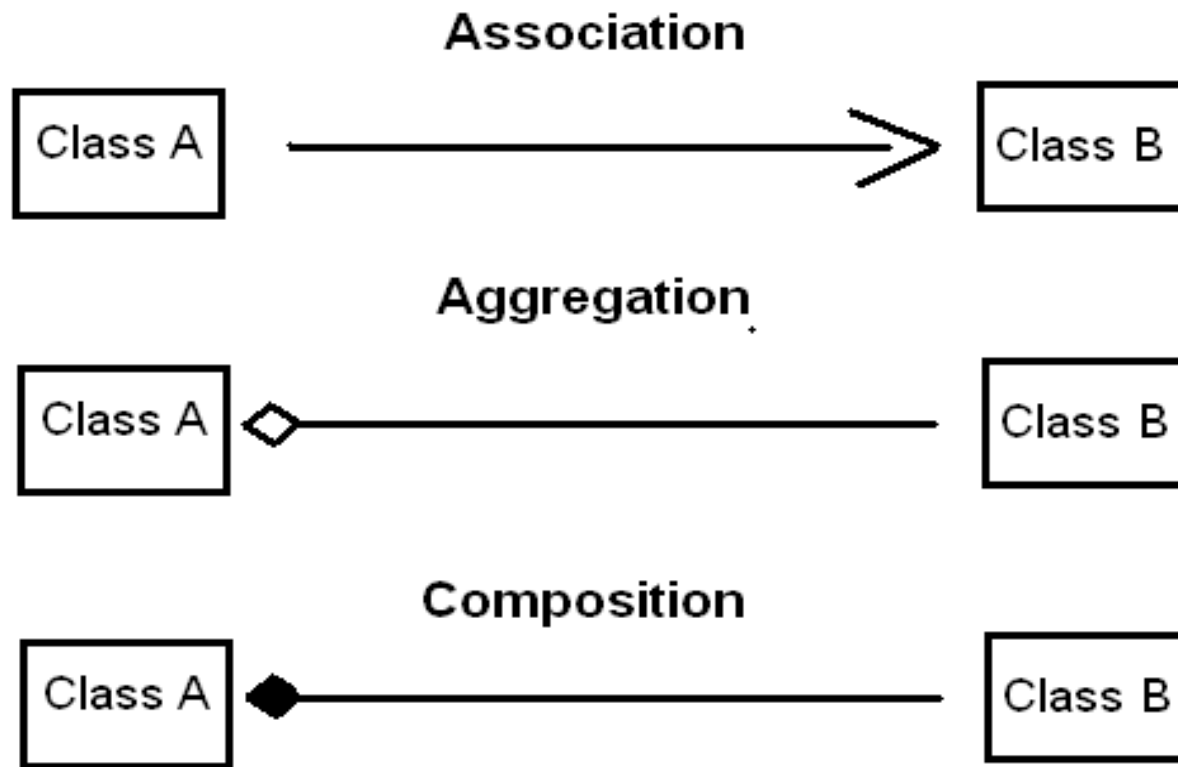
- If a part class cannot exist, without Whole class, then we will call that Association as Composition.
- Life of Whole class and part class will end at the same time.
- e.g. a car has an engine and if the whole car is destroyed the engine cannot function, or a body has a heart and other organs but if the body is destroyed then those organs cannot exist.

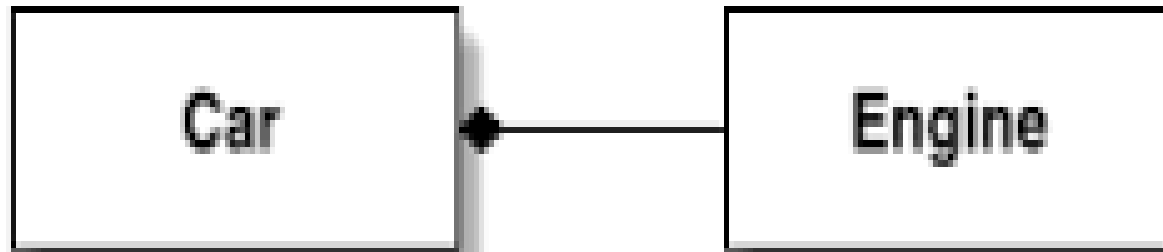


Relationship between Association, Aggregation and composition

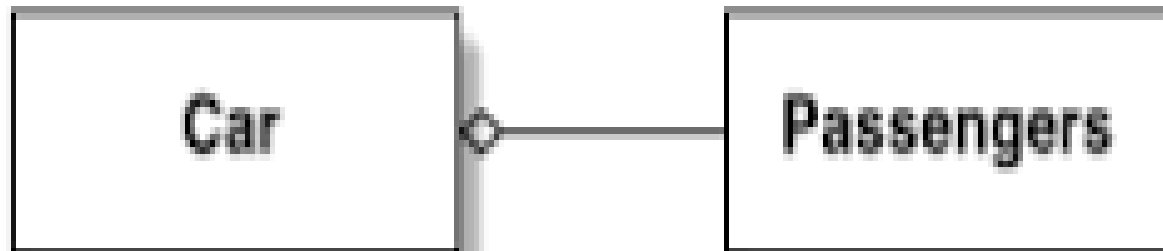


UML representation of an association between classes



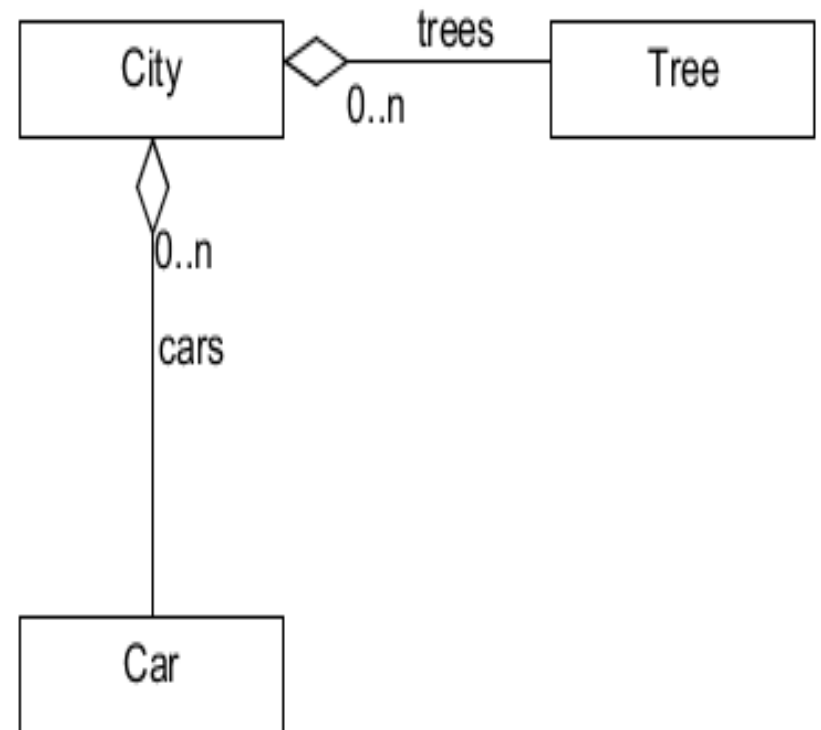
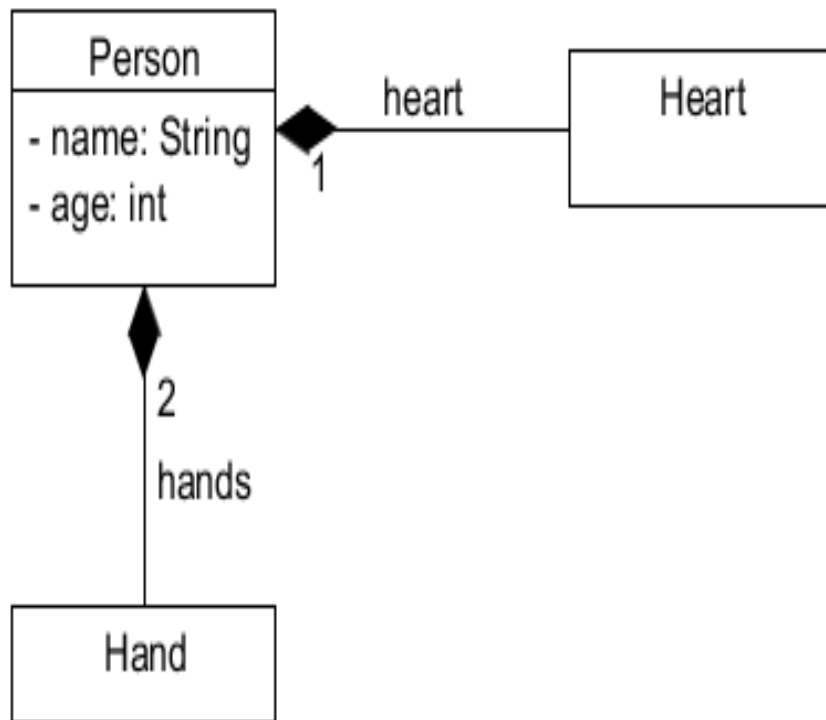


Composition: every car has an engine.



Aggregation: cars may have passengers, they come and go





END

